Universidad Católica del Uruguay

Facultad de Ingeniería y Tecnologías

Ingeniería en Informática

# Informe de Proyecto

Base de Datos 1

Trabajo Obligatorio: Torneo Interfacultades de Fútbol 5

**Integrantes**

Santiago Acosta

Milagros García

Federico Martínez

6 de noviembre de 2026

# Índice

**1. Introducción** **2**

**2. Descripción general de la solución** **2**

2.1. Arquitectura . . . . . 2

2.2. Modelo de datos y datos de prueba . . . . . 3

2.3. Usuarios y funcionalidades . . . . . 3

**3. Decisiones de implementación en la base de datos** **4**

3.1. Torneo, equipos y planteles . . . . . 4

3.2. Partidos . . . . . 6

3.3. Planillas, resultado y sanciones . . . . . 6

3.4. Corrección dentro de las 48 horas . . . . . 7

3.5. Usuarios de la base y permisos . . . . . 7

**4. Decisiones de implementación de la aplicación** **7**

4.1. Backend . . . . . 7

4.2. Frontend . . . . . 8

4.3. Validación en todas las capas . . . . . 8

**5. Reportes y consultas propuestas** **9**

**6. Mejoras al modelo de datos** **9**

6.1. Mejoras implementadas . . . . . 9

6.2. Mejoras consideradas . . . . . 10

**7. Conclusión** **10**

**8. Bibliografía** **11**

# 1. Introducción

El presente informe corresponde al trabajo obligatorio de Base de Datos 1, que consiste en diseñar e implementar un sistema de información para gestionar el Torneo Interfacultades de Fútbol 5 de la universidad. Actualmente la inscripción de los equipos, el armado del fixture y la carga de los resultados se hacen en planillas de cálculo y grupos de mensajería, lo que provoca errores en la tabla de posiciones, partidos superpuestos en una misma cancha y jugadores suspendidos que igual salen a jugar. El sistema busca centralizar esa información y que las reglas del torneo las controle el propio sistema y no las personas que lo organizan.

La solución se compone de una base de datos MySQL, un backend en Python con FastAPI que accede a la base sin ningún ORM y un frontend en React, que se levantan juntos con Docker Compose. A lo largo del informe se describe la solución construida, se fundamentan las decisiones de implementación tomadas tanto en la base como en la aplicación, se explican los reportes junto con las tres consultas propuestas por el equipo y se detallan las mejoras implementadas y consideradas sobre el modelo de datos. La bitácora del trabajo se entrega en un archivo aparte y el instructivo para correr la aplicación se encuentra en el README del repositorio.

# 2. Descripción general de la solución

## 2.1. Arquitectura

El sistema tiene tres servicios. La base de datos es un MySQL 8.0 que, al crearse, ejecuta en orden alfabético los scripts de la carpeta sql del repositorio, que contienen las tablas, los triggers, los datos de prueba, las vistas de los reportes y los usuarios. La API está escrita en Python con FastAPI y ejecuta las consultas con mysql-connector-python, todas escritas a mano y con parámetros. El frontend está hecho en React con Vite y es lo que usan los administradores y los planilleros desde el navegador.

Se eligió FastAPI porque valida los datos de entrada con Pydantic a partir de la definición de cada ruta y porque genera sola la documentación de la API, lo que sirvió para dividir el trabajo, ya que mientras una parte del equipo escribía las rutas otra armaba las pantallas consultando esa documentación. En el frontend se usaron pocas dependencias, react-router-dom para la navegación y Pico.css como base de estilos, sin librerías de componentes ni de manejo de estado.

En Docker Compose la API espera a que la base esté sana, y la base recién se considera sana cuando el usuario de la aplicación puede leer la tabla login, por lo que la API no llega a conectarse mientras la base se está cargando. Además se configuró la zona horaria America/Montevideo en el contenedor de la base. Sin ella MySQL trabaja en UTC, y las

reglas que comparan contra la hora actual, como el plazo de 48 horas para corregir una planilla, quedaban corridas tres horas.

Como la letra pide un script SQL completo, además de los archivos de la carpeta sql se armó `script_completo`.sql, que los reúne en el mismo orden y crea la base desde cero. Se verificó que al cargarlo en un MySQL vacío quedan los mismos datos, usuarios y permisos que con Docker.

## 2.2. Modelo de datos y datos de prueba

El modelo conserva las catorce estructuras de la letra con sus nombres y claves. Las únicas diferencias son que la columna de la contraseña se llama contrasena y que los valores de los ENUM se escribieron sin tildes, para evitar problemas de codificación entre la base, el backend y el navegador. Sobre ese modelo se agregaron un atributo y varias restricciones, que se detallan en la sección 6.

Los datos de prueba cargan seis facultades, cinco turnos, dos complejos con dos canchas cada uno, el torneo del segundo semestre de 2026, 42 jugadores y seis equipos de siete jugadores, uno por facultad. El fixture tiene 16 partidos, de los cuales las fechas 1, 2 y 3 están jugadas con sus planillas, goles y tarjetas, y en la fecha 4 hay un partido postergado y uno cancelado. Los datos se armaron para que aparezcan los casos que el sistema tiene que resolver. Iniesta, de Barsa Sin Plata, recibe una roja en la fecha 1, no puede figurar en la planilla de la fecha 2 y vuelve a jugar en la 3. Abreu, de Nacional Campeón, llega a tres amarillas y queda suspendido para la fecha 4, y Lampard, de Manchester Perifar, tiene una sanción disciplinaria para las fechas 3 y 4. En la fecha 2 Ingecapos y Peñarol Penal empatan 2 a 2 con un gol en contra de Morena, que se le suma a Ingecapos. Con estos datos la tabla queda encabezada por Manchester Perifar con 10 puntos, el goleador es Rooney con 4 goles, y Barsa Sin Plata y Nacional Campeón terminan iguales en puntos y en diferencia de goles, por lo que el orden lo definen los goles a favor.

Como todos los partidos jugados tienen más de 48 horas, otro script pasa el partido entre Ingecapos y Manchester Perifar de la fecha 4 al día anterior a la creación de la base y lo marca como jugado, para poder mostrar la carga de una planilla dentro del plazo.

## 2.3. Usuarios y funcionalidades

El administrador gestiona toda la información del torneo, es decir, los torneos y sus etapas, los jugadores, los equipos y sus planteles, los complejos y canchas, los partidos y las sanciones, y es el único que puede ver los reportes. El planillero solo ve los partidos y la planilla de cada uno, donde marca quiénes jugaron y carga los goles y las tarjetas. También puede marcar como jugado un partido cuyo turno ya terminó, ya que es quien está en la cancha y sin ese paso no podría cargar la planilla.

Cada ABM tiene una pantalla con la tabla de datos y un formulario de alta y modificación. Algunas pantallas muestran las reglas antes de que se llegue al error. La de equipos indica cuántos jugadores y cuántos docentes o funcionarios tiene el plantel y queda en solo lectura cuando el torneo ya no está en inscripción, y la planilla marca a los suspendidos, muestra el marcador calculado con los goles y se bloquea cuando el partido no está jugado o pasaron las 48 horas.

# 3. Decisiones de implementación en la base de datos

Una de las principales decisiones del proyecto fue que todas las reglas de la letra se controlaran en la base de datos, de forma que ninguna aplicación pueda dejar datos inconsistentes aunque tengan un error. Para cada regla se usó la herramienta más simple que alcanzara, empezando por las claves foráneas y los UNIQUE, siguiendo por los CHECK y dejando para el final los triggers, que rechazan la operación con SIGNAL SQLSTATE '45000' y un mensaje en español que después es el que ve el usuario en pantalla. La tabla 1 resume dónde se controla cada regla.

## 3.1. Torneo, equipos y planteles

La regla de que los planteles solo se modifican en inscripción definió cómo se carga un torneo. Toda edición se crea en inscripción, en esa etapa se inscriben los equipos y se arman los planteles, y recién después pasa a en curso. Los triggers de `equipo_jugador_torneo` y el de alta de equipo rechazan la operación si el torneo no está en inscripción. Los datos de prueba siguen el mismo orden y terminan con un UPDATE que pasa el torneo a en curso.

Ese cambio de etapa se aprovecha para validar lo que no se puede controlar fila por fila. El mínimo de cinco jugadores no puede revisarse al insertar, porque todo plantel empieza vacío, y lo mismo pasa con el capitán, porque el equipo se crea antes que su plantel. Por eso el trigger de torneo, al pasar de inscripción a en curso, verifica que todos los equipos tengan al menos cinco jugadores y que cada capitán esté en su plantel. El mismo trigger impide volver a una etapa anterior, ya que de lo contrario alcanzaría con volver a inscripción para modificar los planteles de un torneo en curso.

En el plantel solo se permite modificar el número de camiseta, y para cambiar al jugador o al equipo hay que borrarlo y cargarlo de nuevo, de modo que el alta siempre pase por todas las validaciones del insert. El plantel guarda el `id_torneo` porque la letra lo incluye en su clave y porque es lo que permite el UNIQUE de un jugador por edición. Para que ese valor no pueda ser distinto del torneo del equipo, se agregó una clave foránea compuesta desde el plantel hacia el par (`id_equipo`, `id_torneo`) de equipo.

Tabla 1. Reglas de la letra y dónde se controlan en la base.

| **Regla** | **Dónde se controla** |
|---|---|
| Jugadores de la facultad del equipo, máximo 10 y como mucho 2 docentes o funcionarios | Trigger al insertar en el plantel |
| Mínimo de 5 jugadores y capitán dentro del plantel | Trigger de torneo al cerrar la inscripción |
| Un jugador en un solo equipo por edición | UNIQUE (`id_torneo`, `ci_jugador`) en el plantel |
| Planteles y equipos solo se modifican en inscripción | Triggers de plantel y de equipo |
| Equipos del partido de la misma edición | Claves foráneas compuestas (equipo, torneo) |
| Un equipo no juega contra sí mismo | CHECK en partido |
| Rival repetido salvo cancelado, un partido por equipo y fecha, cancha libre, 5 jugadores por equipo | Triggers de partido |
| Jugado solo si el turno ya pasó y un partido jugado no se borra | Triggers de partido |
| En la planilla solo jugadores del plantel de los dos equipos y no suspendidos | Clave foránea al plantel y trigger de planilla |
| Goles y tarjetas solo de jugadores que están en la planilla | Clave foránea compuesta a `planilla_partido` |
| Resultado calculado a partir de los goles | Vista `resultado_partido` |
| Suspensión automática por roja o tres amarillas | Triggers al insertar una tarjeta |
| Corrección solo dentro de las 48 horas | Triggers de planilla, gol y tarjeta |
| Turnos de una hora entre las 18:00 y las 23:00 | CHECK en turno |

## 3.2. Partidos

Los equipos local y visitante referencian al par (`id_equipo`, `id_torneo`) de equipo junto con el torneo del partido, por lo que no se puede programar un partido entre equipos de ediciones distintas. El resto de las reglas de programación necesita mirar otros partidos y se controla con triggers antes de insertar y de modificar, que llaman a un mismo procedimiento para no repetir las comprobaciones.

En esas comprobaciones se ignoran los partidos cancelados, porque la letra permite volver a programar un cruce cuyo partido fue cancelado y no tendría sentido que ese partido siguiera ocupando una cancha. Para la cancha se evaluó un UNIQUE sobre día, turno, cancha y complejo, pero se descartó justamente porque el partido cancelado habría seguido bloqueando ese lugar. Los datos de prueba incluyen el caso, ya que el partido entre Peñarol Penal y Dream Team Humanista se canceló en la fecha 4 y vuelve a estar programado en la fecha 6. Por último, un partido solo pasa a jugado si el día y la hora de fin de su turno ya pasaron, y un partido jugado no se puede borrar.

## 3.3. Planillas, resultado y sanciones

En la planilla la mayoría de las reglas las resuelven las claves. `planilla_partido` tiene una clave foránea compuesta (`id_equipo`, `ci_jugador`) hacia el plantel, y gol y tarjeta tienen una clave foránea compuesta (`id_partido`, `ci_jugador`) hacia `planilla_partido`, por lo que solo pueden convertir goles o recibir tarjetas los jugadores que figuran en la planilla de ese partido. El trigger de la planilla controla lo que las claves no pueden expresar, que el partido esté jugado, que el equipo sea el local o el visitante y que el jugador no esté suspendido. Un jugador está suspendido si la fecha del partido cae entre `numero_fecha_inicio` y `numero_fecha_inicio` + `cantidad_fechas` - 1 de alguna de sus sanciones en la edición, lo que cubre por igual las sanciones automáticas y las disciplinarias.

Las sanciones automáticas las crean dos triggers después de insertar una tarjeta. Una roja genera una sanción de una fecha que empieza en la fecha siguiente, y con una amarilla se cuentan las amarillas del jugador en la edición y, si la cantidad es múltiplo de tres, se genera la sanción por acumulación. Como la sanción la inserta el trigger, el planillero no necesita permisos sobre la tabla de sanciones. Cabe aclarar que dos amarillas en un mismo partido no se convierten solas en una roja, y si el jugador es expulsado es el planillero el que carga la roja.

El resultado del partido no se guarda en ninguna tabla, sino que se calcula a partir de los goles en la vista `resultado_partido`, que suma cada gol al equipo de su autor según la planilla y los goles en contra al rival. Al no existir un resultado almacenado, no hay forma de que quede distinto de los goles cargados, y cualquier corrección se refleja enseguida en la tabla de posiciones.

### 3.4. Corrección dentro de las 48 horas

Se decidió contar las 48 horas desde el final del turno del partido y aplicar la regla a toda la planilla y no solo a los goles. Ocho triggers rechazan altas, cambios y bajas de goles y tarjetas, y altas y bajas de jugadores en la planilla, una vez que venció el plazo. Si las tarjetas se pudieran cambiar después, se estarían modificando sanciones de fechas que ya se jugaron.

Estos triggers están en el último script a propósito. Si existieran antes de los datos de prueba no se podrían cargar los goles y tarjetas de las fechas jugadas, que tienen varias semanas. Es la misma idea que con el torneo en inscripción, ya que los datos de prueba pasan por todas las demás validaciones y aun así se pueden cargar.

### 3.5. Usuarios de la base y permisos

Se crearon tres usuarios de MySQL. El usuario admin puede leer, insertar, modificar y borrar en todas las tablas, pero no crear ni borrar tablas. El usuario planillero puede leer las tablas que necesita, insertar y borrar en `planilla_partido`, insertar, modificar y borrar en gol y tarjeta, y modificar únicamente la columna estado de partido para marcarlo como jugado, aunque el trigger de partido le sigue impidiendo marcar un partido que todavía no terminó. El tercero, `app_login`, solo puede leer la tabla login y es el que usa el backend para validar el ingreso, cuando todavía no sabe qué rol tiene el usuario.

Al planillero se le dio lectura tabla por tabla para dejar afuera login, que guarda las contraseñas, ya que MySQL no permite excluir una tabla de un permiso sobre toda la base sin activar una opción especial del servidor. Las contraseñas de la aplicación no se guardan en texto plano sino como su hash SHA-256, calculado con la función SHA2 de MySQL.

## 4. Decisiones de implementación de la aplicación

### 4.1. Backend

La decisión más importante del backend fue que cada pedido se conecte a la base con el usuario de MySQL del rol de quien inició sesión. El login devuelve un token JWT con el correo y el rol, que vence a las ocho horas, y con ese rol se elige si la conexión se abre con el usuario admin o con planillero. Las rutas verifican el rol y responden 403 si no corresponde, pero aunque una ruta olvidara hacerlo, la base igual rechazaría lo que el planillero no tiene permitido.

Como las reglas están en la base, el backend no las repite. Cuando un trigger rechaza una

operación, MySQL devuelve el error 1644 con el mensaje del SIGNAL, y un manejador común a toda la API lo devuelve tal cual, por ejemplo “El jugador esta suspendido para esta fecha”. El mismo manejador traduce otros errores frecuentes, como un registro duplicado o un borrado que choca con datos que dependen de ese registro, así que ninguna ruta tiene que atrapar errores por su cuenta. Durante las pruebas se encontró que, cuando el usuario no tiene permisos sobre la tabla relacionada, MySQL devuelve los errores de clave foránea 1216 y 1217 en lugar de 1451 y 1452, y hubo que agregarlos para que no aparecieran como un error genérico.

Cuando una operación necesita varias escrituras que tienen que quedar todas o ninguna, se agrupan en una transacción. Por ejemplo, para borrar un equipo primero se borra su plantel y después el equipo, y si el equipo ya tiene partidos o el torneo no está en inscripción, la operación falla y el plantel queda como estaba. Todas las consultas usan parámetros, y en los reportes el nombre de la vista sale de una lista fija del código y no de la URL, por lo que tampoco por ahí se puede inyectar SQL.

## 4.2. Frontend

Cada pantalla es una página que reutiliza dos componentes genéricos, una tabla para cualquier lista y un formulario de alta y modificación que recibe los campos con sus validaciones. Todas las llamadas al backend pasan por una sola función que agrega el token y convierte los errores en un mensaje para mostrar. Las rutas están protegidas por rol, de modo que el planillero solo ve sus pantallas y, si escribe la dirección de otra, vuelve al inicio. En cuanto al diseño, la aplicación usa la estética de una pizarra de cancha y se adaptó también para pantallas chicas.

## 4.3. Validación en todas las capas

En el frontend se usan los atributos de validación de HTML (required, maxLength, pattern, min y max), con lo que el navegador frena el envío antes de que llegue al servidor. En el backend cada ruta define con Pydantic el formato de lo que recibe, como largos, rangos, patrones y valores posibles de los estados y tipos, y se recortan los espacios de los textos. En la base están los NOT NULL, ENUM, CHECK, claves y triggers ya descritos.

Se buscó que las tres capas apliquen la misma regla. La cédula, por ejemplo, se valida con siete u ocho dígitos en el formulario, en Pydantic y en un CHECK de jugador. Las reglas que dependen de otros datos, como el máximo del plantel o la cancha ocupada, quedan solo en la base, porque repetirlas en el backend obligaría a mantener dos versiones de la misma consulta. En esos casos el frontend evita el error cuando puede, por ejemplo no ofreciendo al mismo equipo como local y como visitante, y si igual ocurre muestra el mensaje del trigger.

# 5. Reportes y consultas propuestas

Todos los reportes se implementaron como vistas de la base. El backend tiene una única ruta que, según el reporte pedido, consulta la vista correspondiente, la filtra por torneo cuando corresponde y la ordena, y por eso las vistas que dependen de la edición incluyen el `id_torneo`. El top 10 de goleadores no se resolvió con un LIMIT dentro de la vista, porque con varias ediciones cargadas el límite cortaría sobre todas juntas, así que se filtra por torneo y se limita desde el backend.

La tabla de posiciones parte de los equipos y no de los partidos, para que un equipo que todavía no jugó aparezca igual con todo en cero. Cada partido de `resultado_partido` aporta una fila por equipo con sus goles a favor y en contra, y con ellas se calculan los partidos jugados, ganados, empatados y perdidos, los goles, la diferencia y los puntos, ordenando por puntos, diferencia y goles a favor. Para los jugadores por facultad y por tipo se dejaron una vista por facultad, otra por tipo y una cruzada, ya que la letra admite las dos lecturas. El resto de los reportes de la letra se implementó tal como se pide.

La primera de las consultas propuestas por el equipo lista los suspendidos para la próxima fecha de cada edición, tomando como próxima la fecha más baja que todavía tiene partidos programados o postergados, junto con el equipo, el motivo y hasta qué fecha dura la sanción. Usa la misma cuenta que el trigger de la planilla, así que muestra a quienes la base no va a dejar entrar en la planilla, y les sirve a los equipos para saber con quién no cuentan. Con los datos de prueba, para la fecha 4 aparecen Kroos por la roja de la fecha 3, Abreu por las tres amarillas y Lampard por la sanción disciplinaria.

La segunda ordena los partidos jugados de la edición según la cantidad total de goles y muestra el resultado de cada uno. La tercera compara el rendimiento de los equipos jugando de local y de visitante, con los partidos ganados, empatados y perdidos, los goles a favor y en contra y los puntos por partido. Como los partidos se juegan en complejos que no pertenecen a ningún equipo, ser local es solo una formalidad, y la consulta permite ver si igual influye en los resultados.

# 6. Mejoras al modelo de datos

## 6.1. Mejoras implementadas

Se agregó el atributo `id_tarjeta` en `sancion_jugador`, que guarda la tarjeta que originó una sanción automática y queda en NULL en las disciplinarias. Su clave foránea tiene ON DELETE CASCADE, por lo que si dentro del plazo se borra una tarjeta cargada por error, la sanción que generó se borra con ella. Sin esa columna no habría forma de saber qué sanción corresponde a qué tarjeta.

Las claves foráneas simples de la letra se reemplazaron por compuestas donde hacía falta. Partido referencia al par (`id_equipo`, `id_torneo`) de equipo, el plantel a ese mismo par, `planilla_partido` al plantel, gol y tarjeta a `planilla_partido`, y `sancion_jugador` al par (`id_torneo`, `ci_jugador`) del plantel, lo que además impide sancionar a alguien que no está inscripto en la edición. Con claves simples todos esos casos inconsistentes eran posibles.

También se agregaron restricciones UNIQUE para que haya una sola edición por año y semestre, para que en cada edición no se repitan nombres de equipo ni haya dos equipos de una misma facultad, para que un jugador esté en un solo plantel por edición, para que no se repitan camisetas en un equipo y para que no haya dos jugadores con el mismo correo. Con CHECK se controla que el semestre sea 1 o 2, que el torneo no termine antes de empezar, que los turnos sean de una hora entre las 18:00 y las 23:00, que la camiseta vaya de 1 a 99, que el minuto de goles y tarjetas vaya de 1 a 60, que la fecha y la cantidad de fechas de una sanción sean al menos 1, y el formato de la cédula y de los correos.

## 6.2. Mejoras consideradas

El hash SHA-256 de las contraseñas no lleva sal, por lo que dos usuarios con la misma contraseña tienen el mismo hash. Lo adecuado sería un algoritmo pensado para contraseñas, como bcrypt, con sal por usuario. Además, la clave para firmar los tokens y las contraseñas de los usuarios de MySQL tienen valores por defecto pensados para correr el proyecto localmente, que en un despliegue real tendrían que venir de variables de entorno.

Una corrección dentro de las 48 horas reemplaza el dato anterior sin dejar rastro, y una tabla de auditoría que registre quién cambió qué y cuándo mejoraría la trazabilidad que pide la letra. En la misma línea, asignar un planillero a cada partido evitaría que cualquier planillero pueda cargar cualquier planilla.

También se consideró generar el fixture de forma automática con un algoritmo de liga a una rueda, en lugar de cargar cada partido a mano, y exigir una cantidad mínima de equipos para que un torneo pase a en curso, algo que hoy no se controla porque la letra no lo pide. Por último, si el volumen de datos creciera mucho, el resultado de cada partido podría guardarse en lugar de calcularse en cada consulta, aunque para un torneo universitario no hace falta.

# 7. Conclusión

El desarrollo del proyecto permitió ver que buena parte de las reglas del torneo se pueden resolver con el diseño de las claves, antes de escribir un solo trigger. Las claves foráneas compuestas son las que impiden que un partido tenga equipos de otra edición, que en la planilla figure alguien que no es del plantel o que haga un gol alguien que no jugó, y los triggers quedaron para las reglas que necesitan mirar otros datos o la hora actual, como

la cancha ocupada, los suspendidos o las 48 horas.

También se vio que las reglas que dependen del tiempo o de la etapa del torneo condicionan cómo se cargan los datos. Decisiones como crear el torneo en inscripción o dejar los triggers de 48 horas en el último script surgieron de tener que cargar datos de prueba que cumplieran con todas las demás reglas, y el orden de los scripts pasó a ser parte del diseño.

Más allá de lo técnico, el proyecto implicó coordinar el trabajo de tres personas sobre el mismo código. El único conflicto de merge del proyecto se dio por dos integrantes editando el mismo archivo de rutas, y a partir de ahí el trabajo se dividió por archivos y se acordó primero la lista de rutas entre el backend y el frontend. Además, probar todo junto contra la base real mostró problemas que por separado no se veían, como los errores 1216 y 1217 o que un equipo con plantel no se podía borrar desde la aplicación.

# 8. Bibliografía


Docker Inc. Docker Compose. `https://docs.docker.com/compose/`

Docker Inc. mysql: Docker Official Image. `https://hub.docker.com/_/mysql`

FastAPI. FastAPI documentation. `https://fastapi.tiangolo.com/`

Oracle Corporation. MySQL 8.0 Reference Manual. `https://dev.mysql.com/doc/refman/8.0/en/`

Oracle Corporation. MySQL Connector/Python Developer Guide. `https://dev.mysql.com/doc/connector-python/en/`

Pico CSS. Pico CSS documentation. `https://picocss.com/docs`

Pydantic. Pydantic documentation. `https://docs.pydantic.dev/`

PyJWT. PyJWT documentation. `https://pyjwt.readthedocs.io/`

React. React documentation. `https://react.dev/`

React Router. React Router documentation. `https://reactrouter.com/`

Universidad Católica del Uruguay. (2026). Trabajo Obligatorio: Sistema para Gestión del Torneo Interfacultades de Fútbol 5. Base de Datos 1, segundo semestre 2026.

Vecteezy. Soccer sports jersey vectors. `https://www.vecteezy.com/`

Vite. Vite documentation. `https://vite.dev/`